GRADE 12 · TERM 3

Exam Countdown: Consolidation for the Final Exam

A pencil-and-paper workbook: practise the exam question types, revise the theory in full sentences, and plan your last weeks.

Name: _____

Class: _____ **Date:** _____

How to use this workbook

1. Work with a **pencil**. Mistakes are how we learn: rub out and try again.
2. Start each unit with **Think First**. Try it on your own before you read the explanation.
3. Read **Learn It**, then do the activities. Trace tables and code are written **by hand**.
4. At the end of each unit, tick the "I can" boxes you are sure of. Write down anything you still want to ask.
5. The **checkpoint test** is done under test conditions, with no notes.

In this workbook

1. Recursion on paper
2. Strings, loops and decisions: exam practice
3. Arrays and text files: exam practice
4. OOP: exam practice
5. 2D arrays: exam practice
6. SQL and databases: exam practice
7. Paper 2 theory: internet, networks and security
8. Exam technique and your study plan
9. Term checkpoint test

> **Matches the Annual Teaching Plan:** Term 3 — consolidation before the final exam: recursion, strings and loops, arrays and text files, OOP, 2D arrays, SQL with joins and GROUP BY; internet, networks and security theory; exam technique.

UNIT 1 Recursion on paper

A recursive method solves a problem by handing a slightly smaller copy of the same problem to itself. In this unit you will watch the calls pile up on paper, follow the answers back, and write recursive methods of your own.

By the end of this unit I can: (tick each box when you are sure)

- ☐ explain the base case and the recursive case in my own words
- ☐ draw the calls of a recursive function as a stack or a tree
- ☐ trace a recursive function from the first call to the final answer
- ☐ spot a missing base case that causes infinite recursion
- ☐ write a recursive function for a new problem

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

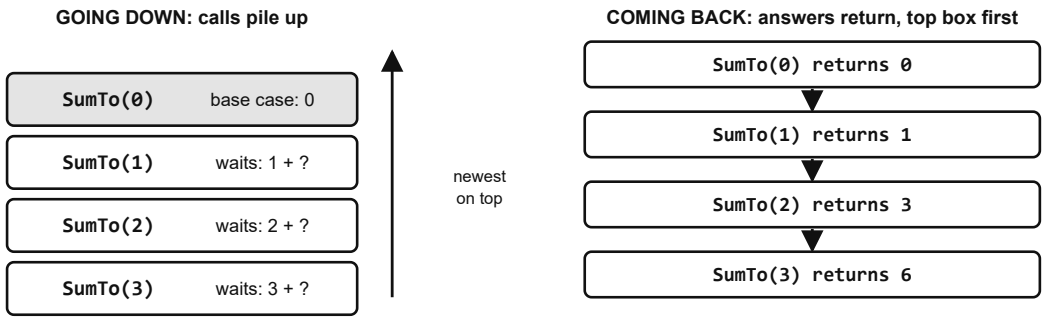
1.1 The queue you cannot see past

You are somewhere in a long queue at the clinic and cannot see the front. You want to know your position, but you may only speak to the person **directly in front of you**, and they may only speak to the person in front of them. Write down what each person says and does so that you finally learn your position. What does the person at the very front say? What does everyone else do with the answer they get?

LEARN IT

Two parts and one rule

Every recursive method has two parts. The **base case** is the smallest problem, answered directly with **no further call**. The **recursive case** calls the same method with a **smaller** problem and then uses the answer. The one rule: every call must move closer to the base case, otherwise the calls never stop. Each call gets its **own** copy of the parameters, and the calls wait in a pile called the **call stack**: the newest call sits on top and is the first to finish.



Delphi

```
function SumTo(n: Integer): Integer;
begin
  if n = 0 then
    Result := 0                  // base case: no further call
  else
    Result := n + SumTo(n - 1);  // recursive case: smaller problem
end;
```

LEARN IT

Down and back up

Code placed **before** the recursive call runs on the way **down**, in the order of the calls; code placed **after** it runs on the way **back up**, in the reverse order. If the base case is missing, or the call does not get smaller, the calls never stop and the program ends with a **stack overflow**. A loop uses less memory, but recursion is natural when a problem contains a smaller copy of itself, such as the digits of a number or the letters of a word. Example: Fall(3) writes down 1, down 2, down 3 because the writing happens after the call.

Delphi

```
procedure Climb(n: Integer);
begin
  if n > 0 then
  begin
    memOut.Lines.Add('up ' + IntToStr(n));    // BEFORE the call
    Climb(n - 1);
  end;
end;                                           // Climb(3): up 3, up 2, up 1

procedure Fall(n: Integer);
begin
  if n > 0 then
  begin
    Fall(n - 1);
    memOut.Lines.Add('down ' + IntToStr(n));  // AFTER the call
  end;
end;                                           // Fall(3): down 1, down 2, down 3
```

1.2 Be the computer: SumTo(5) [5]

Work **down** the rows to find the base case, then fill in the value returned, starting at the **bottom row** and working up.

Delphi

```
function SumTo(n: Integer): Integer;
begin
  if n = 0 then
    Result := 0
  else
    Result := n + SumTo(n - 1);
end;

iAns := SumTo(5);
```

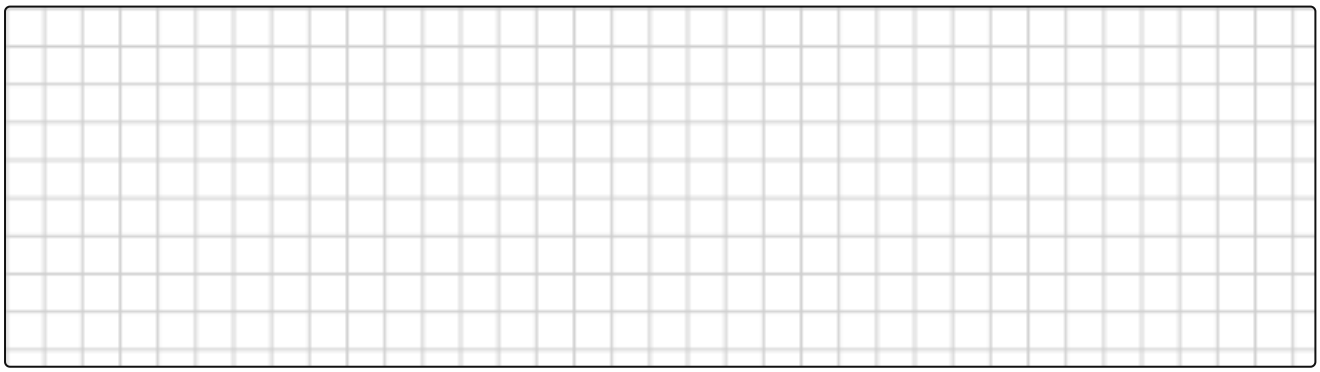
Call	n	Base case? (Yes/No)	Value returned
SumTo(5)	5		
SumTo(4)	4		
SumTo(3)	3		
SumTo(2)	2		
SumTo(1)	1		
SumTo(0)	0		

1.3 Draw the call tree: Fib(4) [4]

This function makes **two** recursive calls. Draw a tree of every call to **Fib(4)**: write the call in a circle and join it to the two calls it makes. Write the value in each circle, then say how many calls were made altogether.

Delphi

```
function Fib(n: Integer): Integer;
begin
  if n <= 1 then
    Result := n
  else
    Result := Fib(n - 1) + Fib(n - 2);
end;
```



1.4 Debug: the function that never stops [3]

Triple(n) should return 3 × n by adding 3 repeatedly, so Triple(4) = 12. Instead the program crashes. Find **two** errors, say what happens when Triple(4) is called, and write the corrected line(s).

Delphi

```
function Triple(n: Integer): Integer;
begin
  if n = 0 then
    Result := 0
  else
    Result := 3 + Triple(n + 1);
end;
```

1.5 Write it: two recursive functions [9]

Write **two** recursive functions. (a) **Multiply(a, b: Integer): Integer** multiplies two whole numbers using **only addition** (b is never negative): Multiply(7, 4) = 7 + 7 + 7 + 7 = 28. (b) **CountChar(sText: string; cFind: Char): Integer** returns how many times cFind appears in sText; pass on *Copy(sText, 2, Length(sText) - 1)* so that the text gets shorter. CountChar('banana', 'a') = 3.

UNIT 2 Strings, loops and decisions: exam practice

The first question in Paper 1 mixes strings, loops and decisions in one short story. Marks come from careful reading, a tidy plan and tracing your own code. This unit gives you four types of practice: trace, predict, debug and write.

By the end of this unit I can: (tick each box when you are sure)

- ☐ use Length, Pos, Copy, Delete, UpperCase and Ord to take a string apart
- ☐ trace a loop that changes a string or a number
- ☐ write decisions that handle boundary values correctly
- ☐ find the errors in code that reads and checks text
- ☐ write a complete loop with validation, counters and a guard against dividing by zero

One thing I want to ask my teacher: _____

2.1 Be the string

Put a pencil on box 1 and walk along the letters as a loop would. (a) How many **vowels** are there? (b) At which position is the space? (c) What would *Copy(sText, 6, 3)* give? (d) Write the string backwards.

1	2	3	4	5	6	7	8	9	10	11	12	13	14
P	O	R	T		E	L	I	Z	A	B	E	T	H

THINK FIRST · no computer needed

2.2 The museum ticket

Rule: children under 13 pay R20; learners aged 13 to 17 pay R35; everyone else pays R50; but **anyone aged 60 or older** pays R25. Write the fee for each age. Then circle the ages that are **boundary values** (the first or last age of a group).

Age	12	13	17	18	59	60	75
Fee (R)							

Why is it dangerous to test a rule like this only with ages such as 8, 25 and 70?

LEARN IT

The Question 1 toolkit

Most Question 1 marks come from a handful of patterns. Learn to recognise them.

Pattern	How it works
Walk a string	*for i := 1 to Length(s) do* and look at *s[i]*. The first character is at position 1.
Take it apart	*Pos(sub, s)* gives the position of the first match, or **0** if there is none. *Copy(s, start, count)* returns text; *Delete(s, start, count)* removes it from s itself (a procedure, not a function).

Characters and codes	*Ord('A')* = 65, *Chr(66)* = 'B'; a lower-case letter is 32 more than its capital. *UpperCase* and *LowerCase* change a whole string, *UpCase* a single character.
Counter and total	Set to 0 **before** the loop. *Inc(iCount)* when the condition is true; add to the total on every pass.
Highest or lowest	Start the highest at a value no mark can be below (or at the first value) and replace it whenever something larger is found.
Reading until a sentinel	Read once **before** a WHILE loop and read again as the **last** line inside it, otherwise the loop never ends.
Decisions	Test the most specific rule first in an else-if ladder, put brackets round every comparison joined by and/or, and use *case* for whole-number ranges such as *80..100*.
Safe division	Test that the counter is more than 0 before dividing by it.

2.3 String functions [7]

Let sText be 'Cape Town'. Length(sText) = _____. Pos('T', sText) = _____. Copy(sText, 6, 4) = _____. Ord('C') = _____. Chr(Ord('c') + 1) = _____. The last character is sText[_____], which is the letter _____. After Delete(sText, 1, 5) the variable sText holds _____.

2.4 Be the computer: splitting words [6]

The text is **'load shedding is back'** (words separated by one space). Complete a row for every check of the WHILE condition; the last two rows show where the loop ends and the final Inc.

```delphi
sText := 'load shedding is back';
iWords := 0;
while Pos(' ', sText) > 0 do
begin
  iPos := Pos(' ', sText);
  sWord := Copy(sText, 1, iPos - 1);
  Delete(sText, 1, iPos);
  Inc(iWords);
end;
Inc(iWords);
```

Check	iPos (0 if none)	sWord	sText after the body	iWords
1				
2				
3				
4				
after the loop: Inc(iWords)				

2.5 Debug: vowel percentage [4]

This code should show what percentage of the characters in **edtText** are vowels (capital or small). Find **four** errors.

Delphi

```
sText := edtText.Text;
iVowels := 0;
for i := 0 to Length(sText) do
  if sText[i] in ['a', 'e', 'i', 'o', 'u'] then
    Inc(iVowels);
rPerc := iVowels / Length(sText) * 100;
lblOut.Caption := rPerc + '%';
```

2.6 Write it: a school e-mail address [6]

A new learner's full name, such as **Zola Ndlovu**, is in **edtName**. Build the e-mail address from the **first letter of the first name**, the **surname** and *@bhs.edu.za*, all in small letters, and show it in **lblEmail**. (Zola Ndlovu gives *zndlovu@bhs.edu.za*.) If there is no space in the name, show a message instead.

2.7 Write it: marks until the sentinel **[9]**

Use an InputBox in a loop to read marks until the user types **-1**. A mark outside 0 to 100 must give a message and must **not** be counted. At the end show in **lblOut** how many valid marks were entered, how many were **80 or more**, and the **highest** mark. If no valid mark was entered, show *No marks*.

UNIT 3 Arrays and text files: exam practice

Question 2 of Paper 1 nearly always reads a text file into arrays, then asks for a report, a search or a sort. The steps never change, so practise them until they feel automatic.

By the end of this unit I can: (tick each box when you are sure)

- ☐ read a comma-separated text file into parallel arrays
- ☐ trace a file-reading loop and a sort of parallel arrays
- ☐ search an array with a Boolean flag
- ☐ write a report from arrays (total, highest, count, average)
- ☐ describe what Reset, Rewrite and Append do and when to use each

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

3.1 Two strips of paper

A park-run results list is kept on **two strips of paper**: names on one strip and finishing times (in seconds) on the other, in the same order. Here are six runners:

Position in list	1	2	3	4	5	6
Name	Thandi	Pieter	Ayesha	Sipho	Lize	Kabelo
Time (s)	1425	1310	1688	1502	1397	1735

(a) Write the names from the **fastest to the slowest**. (b) A helper puts only the **times strip** in order and leaves the names strip alone. Whose time would now be next to Thandi's name? What must happen whenever you move a time?

LEARN IT

Question 2 in four steps

1. Declare parallel arrays (one per field, same index) and a counter. **2. Read**: AssignFile, check FileExists, Reset, then *while not EOF* read a line, split it and store the pieces at the next index, and finally CloseFile. **3. Process** with loops: total, highest, count, search or sort. **4. Display** the answer with IntToStr or FloatToStrF. **Reset** opens a file to read, **Rewrite** creates a new file (and wipes an old one!) and **Append** adds to the end. The function in the next Learn It shows a time in seconds as minutes and seconds. Never use LoadFromFile or SaveToFile: the examiners want the loop.

Parkrun.txt (eight lines)

```
Thandi Nkosi,F,1425
Pieter Visser,M,1310
Ayesha Cassiem,F,1688
Sipho Zulu,M,1502
Lize Botha,F,1397
Kabelo Molefe,M,1735
Nomsa Dube,F,1520
Riaan Pretorius,M,1290
```

Delphi

```
var
  arrName   : array[1..100] of string;
  arrGender : array[1..100] of string;
  arrTime   : array[1..100] of Integer;
  iCount    : Integer;

AssignFile(tFile, 'Parkrun.txt');
Reset(tFile);
while not EOF(tFile) do
begin
  ReadLn(tFile, sLine);
  Inc(iCount);
  iPos := Pos(',', sLine);
  arrName[iCount] := Copy(sLine, 1, iPos - 1);
  Delete(sLine, 1, iPos);
  iPos := Pos(',', sLine);
  arrGender[iCount] := Copy(sLine, 1, iPos - 1);
  arrTime[iCount] := StrToInt(Copy(sLine, iPos + 1, Length(sLine)));
end;
CloseFile(tFile);
```

LEARN IT

Search, sort and a helper

A **linear search** uses a flag and stops as soon as the item is found. A **sort** must swap **every** parallel array whenever it swaps two items. Binary search only works on a sorted array. The helper function turns seconds into *m:ss*, adding a leading 0 when the seconds are less than 10.

Delphi

```
bFound := False;
i := 1;
while (i <= iCount) and (not bFound) do
  if arrName[i] = sFind then
    bFound := True
  else
    Inc(i);
// bFound False means not found; otherwise i is the position

// bubble sort, smallest time first, names move with the times
for i := 1 to iCount - 1 do
  for j := 1 to iCount - i do
    if arrTime[j] > arrTime[j + 1] then
    begin
      iTemp := arrTime[j];  arrTime[j] := arrTime[j + 1];  arrTime[j + 1] := iTemp;
      sTemp := arrName[j];  arrName[j] := arrName[j + 1];  arrName[j + 1] := sTemp;
    end;

function MinSec(iTime: Integer): string;
var
  iSec: Integer;
begin
  iSec := iTime mod 60;
  Result := IntToStr(iTime div 60) + ':';
  if iSec < 10 then
    Result := Result + '0';
  Result := Result + IntToStr(iSec);
end;
```

3.2 Be the computer: reading the file [6]

Parkrun.txt is printed in *Question 2 in four steps*. Trace the loop for the **first three lines**: Thandi Nkosi,F,1425; Pieter Visser,M,1310; Ayesha Cassiem,F,1688. Write the values **after** each pass. The row for check 4 shows the loop ending.

Delphi

```
iTotal := 0;  iFem := 0;
AssignFile(tFile, 'Parkrun.txt');
Reset(tFile);
while not EOF(tFile) do
begin
  ReadLn(tFile, sLine);
  iPos := Pos(',', sLine);
  sName := Copy(sLine, 1, iPos - 1);
  Delete(sLine, 1, iPos);
  iPos := Pos(',', sLine);
  sGender := Copy(sLine, 1, iPos - 1);
  iTime := StrToInt(Copy(sLine, iPos + 1, Length(sLine)));
  iTotal := iTotal + iTime;
  if sGender = 'F' then
    Inc(iFem);
end;
// (the file has only these three lines in this question)
```

Check	EOF(tFile)?	sName	sGender	iTime	iTotal	iFem
1	False					
2	False					
3	False					
4	True					

3.3 Be the computer: sorting parallel arrays [5]

arrName holds Sipho, Riaan, Ayesha, Lize, Pieter and **arrTime** holds 1502, 1290, 1688, 1397, 1310. Write both arrays after each pass of the bubble sort in Learn It (smallest time first) and the number of swaps.

Delphi

```
for i := 1 to 4 do
  for j := 1 to 5 - i do
    if arrTime[j] > arrTime[j + 1] then
    begin
      // swap arrTime[j] with arrTime[j + 1]
      // swap arrName[j] with arrName[j + 1]
    end;
```

Pass (i)	arrTime after the pass	arrName after the pass	Swaps
1			
2			
3			
4			

3.4 Write it: file into arrays [10]

Write the code that reads **Parkrun.txt** into the arrays declared in Learn It and counts the runners in **iCount**. Test that the file exists first and show a message if it does not. (tFile, sLine and iPos are declared; the first line of the file is a runner, not a heading.)

3.5 Write it: the report [7]

The arrays are filled and **iCount** holds the number of runners. Show the **fastest runner's name and time** (as m:ss) in **lblFast**, how many runners finished in **under 25 minutes** (1 500 seconds) in **lblUnder**, and the **average time** (as m:ss, whole seconds) in **lblAvg**. Use the function MinSec.

UNIT 4 OOP: exam practice

Question 3 gives you a short story and asks for a class: attributes, a constructor, accessors, mutators, auxiliary methods and a ToString, then code in the form that uses the objects. Learn the pattern once and it will repeat in every paper.

By the end of this unit I can: (tick each box when you are sure)

- ☐ write a class with a constructor, accessor, mutator, auxiliary and ToString method
- ☐ explain private and public in my own words
- ☐ trace what happens to an object's attributes call by call
- ☐ explain what happens when one object variable is assigned to another
- ☐ use an object from the form and see how two classes can work together

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

4.1 Blueprint for a parcel

A courier company stores, for each parcel, an **ID code**, its **weight in kg**, a **delivery zone** (1, 2 or 3) and whether it is **insured**. The price depends on the zone, the weight over 2 kg and the insurance. On the lines below plan a class **TParcel**: (a) list the private attributes with their data types and (b) list the methods you would need, grouped as *constructor*, *accessors*, *mutators*, *auxiliary* and *other*. Why should the attributes be private?

LEARN IT

A complete class: TParcel (the interface)

A class has **private attributes** (fields, prefixed with f) and **public methods**. Put each class in its own unit (here UParcel) so that other units can only use the public methods; the interface lists the attributes and the headings of the methods.

Delphi

```
unit UParcel;
interface
type
  TParcel = class
  private
    fID      : string;
    fWeight  : Real;
    fZone    : Integer;            // 1, 2 or 3
    fInsured : Boolean;
    function BaseRate : Real;      // auxiliary: used only inside the class
  public
    constructor Create(sID: string; rWeight: Real; iZone: Integer);
    function    GetWeight : Real;
    function    GetZone : Integer;
    procedure   SetZone(iNew: Integer);
    procedure   SetWeight(rNew: Real);
    procedure   SetInsured(bNew: Boolean);
    function    CalcCost : Real;
    function    ToString : string;
  end;
```

LEARN IT

TParcel: constructor, accessor and mutator

The **constructor** sets the starting values (here insured starts False without being a parameter). An **accessor** (Get) returns an attribute. A **mutator** (Set) changes one and can protect it with a rule.

Delphi

```
implementation
constructor TParcel.Create(sID: string; rWeight: Real; iZone: Integer);
begin
  fID := sID;
  fWeight := rWeight;
  fZone := iZone;
  fInsured := False;               // a default, not a parameter
end;
procedure TParcel.SetZone(iNew: Integer);
begin
  if (iNew >= 1) and (iNew <= 3) then
    fZone := iNew;                 // the mutator protects the attribute
end;
```

LEARN IT

TParcel: auxiliary method, CalcCost and ToString

An **auxiliary** method is a private helper that a public method calls; it returns a result and never changes an attribute. **CalcCost** works from attributes without storing the answer. **ToString** returns the object's state as text. In the form, create the object with *objP := TParcel.Create('PX7', 3.5, 2);* and call the methods on the **object variable**, for example *objP.SetInsured(True)* or *memOut.Lines.Add(objP.ToString)*; with PX7 insured, ToString ends in R88.75.

Delphi

```
function TParcel.BaseRate : Real;
begin
  case fZone of
    1: Result := 35;
    2: Result := 55;
  else
    Result := 80;
  end;
end;

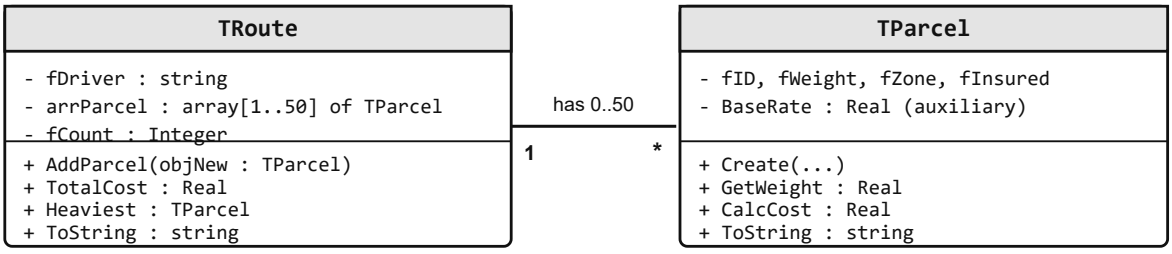
function TParcel.CalcCost : Real;
begin
  Result := BaseRate;                          // the auxiliary method gives the zone rate
  if fWeight > 2 then
    Result := Result + (fWeight - 2) * 12.5;   // R12.50 for every kg over 2
  if fInsured then
    Result := Result + 15;
end;

function TParcel.ToString : string;
begin
  Result := fID + ': ' + FloatToStrF(fWeight, ffFixed, 6, 1) + 'kg, zone ' + IntToStr(fZone)
            + ', R' + FloatToStrF(CalcCost, ffFixed, 8, 2);
end;
```

LEARN IT

Two classes working together

A class can hold objects of another class. A **TRoute** keeps an array of TParcel objects and uses their methods. Each parcel calculates its own cost, so the route only has to add them up. A method may also **return an object**: *Heaviest* returns the TParcel with the greatest weight.



Delphi

```
function TRoute.TotalCost : Real;
var
  i : Integer;
begin
  Result := 0;
  for i := 1 to fCount do
    Result := Result + arrParcel[i].CalcCost;   // each object answers for itself
end;
```

4.2 Be the computer: an object's attributes [8]

Complete the table after each statement. The class is TParcel above: *SetZone* accepts only 1 to 3, *SetWeight* only a weight above 0. CalcCost is the price the method gives *at that moment*, to 2 decimal places.

Delphi

```
objP := TParcel.Create('PX7', 3.5, 2);
objP.SetInsured(True);
objP.SetZone(3);
objP.SetZone(5);
objP.SetWeight(1.5);
objP.SetWeight(-2);
```

After this statement	fWeight	fZone	fInsured	CalcCost (R)
objP := TParcel.Create('PX7', 3.5, 2);				
objP.SetInsured(True);				
objP.SetZone(3);				
objP.SetZone(5);				
objP.SetWeight(1.5);				
objP.SetWeight(-2);				

4.3 Predict: two variables, one object [4]

Write the three lines that are added to **memOut**. (Use the TParcel class above: each ToString shows ID, weight, zone and cost.)

Delphi

```
objA := TParcel.Create('A1', 1.0, 1);
objB := TParcel.Create('B2', 4.0, 3);
objC := objB;
objC.SetZone(1);
objA.SetInsured(True);
memOut.Lines.Add(objA.ToString);
memOut.Lines.Add(objB.ToString);
memOut.Lines.Add(objC.ToString);
```

4.4 Write it: complete the class [7]

A spaza shop keeps one **TItem** object for each product. The interface is below; **GetQty**, **Restock**, **SetPrice** and the private **StockValue** (price × quantity) are already written. Write the implementation of: the **constructor** (sets the name and price and starts the quantity at 0); **Sell**, which reduces the quantity and returns True **only if there is enough stock** (otherwise nothing changes and it returns False); and **ToString**, which uses StockValue to give text such as *Maize meal: 6 @ R42.50 (value R255.00)*.

```Delphi
type
  TItem = class
  private
    fName  : string;
    fPrice : Real;
    fQty   : Integer;
    function StockValue : Real;
  public
    constructor Create(sName: string; rPrice: Real);
    function    GetQty : Integer;
    procedure   SetPrice(rNew: Real);
    procedure   Restock(iAdd: Integer);
    function    Sell(iHowMany: Integer) : Boolean;
    function    ToString : string;
  end;
```

4.5 Write it: use the object in the form **[6]**

Objects are made in the form. **objItem** is declared at form level as TItem. (a) In **btnNew**: create the object from **edtName** and **edtPrice** (a decimal number), freeing any old object first. (b) In **btnSell**: call Sell with the value of **sedQty**; show *Not enough stock* in lblMsg if it returns False, otherwise show the item's ToString in lblMsg.

UNIT 5 2D arrays: exam practice

A 2D array is a table in memory: the first index chooses the row and the second chooses the column. Exam questions on grids all come down to three skills: knowing which index to fix, writing nested loops in the right order, and checking the edges of the grid.

By the end of this unit I can: (tick each box when you are sure)

- ☐ declare a 2D array and visit every cell with nested loops
- ☐ total a row, a column or the whole grid
- ☐ find the highest value and remember where it is
- ☐ search a grid and stop when the item is found
- ☐ trace nested loops that keep a running total and a best-so-far value

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

5.1 The seat plan

A small hall has 5 rows and 8 seats per row. X means **booked** and O means **free**.

	c = 1	c = 2	c = 3	c = 4	c = 5	c = 6	c = 7	c = 8
r = 1	X	X	O	O	X	X	X	O
r = 2	O	X	O	X	O	O	X	O
r = 3	X	O	X	X	O	X	O	O
r = 4	X	O	O	O	O	X	O	X
r = 5	X	X	X	O	O	O	X	O

(a) How many seats are booked? (b) Which row has the most booked seats? (c) Find the **first row** that has **three free seats side by side** and write its row and the column where the three seats start. (d) Describe in words the steps your eyes follow for (c).

THINK FIRST · no computer needed

5.2 Which way does the total go?

Five learners wrote four tests:

Learner	Test 1	Test 2	Test 3	Test 4
Amahle	62	70	58	74
Brandon	45	52	60	48
Chantelle	81	77	90	85
Dumisa	55	63	49	58
Elmarie	70	66	72	68

(a) To get **Chantelle's** total, which cells do you add? Does the row number or the column number stay the same? (b) To get the **class total for Test 3**, which cells do you add, and which number stays the same? (c) How many cells does a loop visit if it adds **every** mark?

LEARN IT

Two indexes, two loops

arrMark[r, c]: **r** is the row, **c** the column. For a **row total** fix the row and let the column change; for a **column total** fix the column and let the row change. Reset a total to 0 **before** the loop that builds it (for a row total, inside the outer loop). The outer loop is the one that changes more slowly. To remember *where* a highest value is, store the row or column number, not only the value. A parallel 1D array (such as *arrName[r]*) gives the name that belongs to row r. To **search**, use a flag and stop as soon as the item is found; to find a run of items side by side, keep a counter that goes up for a match and goes back to 0 for a non-match. To **display** a row, build one string with the inner loop and add it to the memo after the inner loop ends.

Delphi

```
var
  arrMark : array[1..5, 1..4] of Integer;       // 5 learners, 4 tests
  arrName : array[1..5] of string;

// row total for every learner
for r := 1 to 5 do
begin
  iRowTotal := 0;                  // reset for each row
  for c := 1 to 4 do
    iRowTotal := iRowTotal + arrMark[r, c];
  memOut.Lines.Add(arrName[r] + ': ' + IntToStr(iRowTotal));
end;

// column total for test 3
iColTotal := 0;
for r := 1 to 5 do
  iColTotal := iColTotal + arrMark[r, 3];
```

5.3 Be the computer: the best learner [5]

The marks table is the one in Think First (Amahle 62, 70, 58, 74; Brandon 45, 52, 60, 48; Chantelle 81, 77, 90, 85; Dumisa 55, 63, 49, 58; Elmarie 70, 66, 72, 68). Write the values **after** each pass of the outer loop.

Delphi

```
iBest := 1;
iBestTotal := 0;
for r := 1 to 5 do
begin
  iRowTotal := 0;
  for c := 1 to 4 do
    iRowTotal := iRowTotal + arrMark[r, c];
  if iRowTotal > iBestTotal then
  begin
    iBestTotal := iRowTotal;
    iBest := r;
  end;
end;
```

r	iRowTotal	iBestTotal	iBest
1			
2			
3			
4			
5			

5.4 Debug: the learner averages [4]

This code should show each learner's average over the four tests (to 1 decimal) in **memOut**. Find **four** errors.

Delphi

```
iTotal := 0;
for r := 1 to 5 do
begin
  for c := 1 to 4 do
    iTotal := iTotal + arrMark[c, r];
  rAvg := iTotal / 5;
  memOut.Lines.Add(arrName[r] + ' ' + rAvg);
end;
```

5.5 Write it: three seats together [6]

A family needs **three free seats side by side** in one row. Search the seat plan row by row and show in **lblFind** the row and the first column of the **first** such group (for example *Row 4, seat 2*), or *None free*.

5.6 Write it: learners and tests [8]

Use **arrMark[1..5, 1..4]** and **arrName[1..5]** (the marks table in Think First). (a) Show every learner's name and **average** (one decimal) in **memOut**. (b) Find the test with the **highest class average** and show *Test 4* (its number) in **lblBest**.

UNIT 6 SQL and databases: exam practice

Question 4 gives you a small extract of a database and asks you to read, write and build queries. The marks come from three habits: running the query by hand in the right order, joining tables with a condition, and writing the SQL inside Delphi correctly.

By the end of this unit I can: (tick each box when you are sure)

- ☐ run a query by hand on an extract of a table and write the result
- ☐ write queries with WHERE, ORDER BY, TOP, dates and calculated fields
- ☐ write a join with GROUP BY and HAVING
- ☐ write INSERT, UPDATE and DELETE statements
- ☐ build and run SQL in Delphi and read the answer record by record

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

6.1 You are the database

A bakery chain keeps its orders in a database. Here is an **extract** that you will use for the whole unit (keep this page open):

tblBranch

BranchID	BranchName	Town
1	Main Street	Bellville
2	Station Road	Parow
3	Vineyard Lane	Durbanville
4	Hilltop	Goodwood

tblOrder

OrderID	BranchID	OrderDate	Product	Qty	UnitPrice
1	1	2026/08/03	Bread	40	18
2	2	2026/08/03	Rusks	12	45
3	1	2026/08/05	Milk tart	6	60
4	3	2026/08/07	Bread	25	18
5	4	2026/08/10	Koeksister	48	4.5
6	2	2026/08/12	Bread	31	18
7	3	2026/08/14	Milk tart	4	60
8	1	2026/09/02	Rusks	20	45
9	4	2026/09/04	Bread	22	18
10	2	2026/09/09	Koeksister	36	4.5
11	3	2026/09/11	Rusks	15	45
12	1	2026/09/15	Milk tart	8	60

(a) How many orders have a Qty of **20 or more**? (b) What is the total Qty of all the **Bread** orders? (c) Order 9 belongs to which **branch name and town**? Which table gave you the answer, and how did you link it to the other?

(d) tblBranch has 4 rows and tblOrder has 12. If a program paired **every** branch with **every** order, how many pairs would that make, and which condition keeps only the sensible pairs?

LEARN IT

The order the database thinks in

A query is not worked out in the order it is written. The database works in this order: **FROM** (which tables), **WHERE** (keep or drop single records, including the join condition), **GROUP BY** (collect the records into groups), **HAVING** (keep or drop whole groups), **SELECT** (work out the columns and the aggregates) and **ORDER BY** (sort). That is why an aggregate such as SUM can never be in WHERE (the groups do not exist yet) and why every field in SELECT that is **not** inside an aggregate must also be in GROUP BY. Use the **join condition** in WHERE whenever two tables are listed in FROM. AND is worked out before OR, so use brackets. Dates are written *#yyyy/mm/dd#*, and to cover a whole month ask for >= the first day and < the first day of the next month. Text values go in double quotes, *&* joins pieces of text, *TOP n* follows SELECT, and for LIKE use the wildcard character that the question's database uses (% or *). In Access you cannot use an alias you created with AS inside WHERE or HAVING: repeat the calculation.

SQL

```sql
SELECT B.BranchName, SUM(O.Qty * O.UnitPrice) AS Revenue
FROM tblBranch B, tblOrder O
WHERE B.BranchID = O.BranchID
AND O.OrderDate >= #2026/09/01#
GROUP BY B.BranchName
HAVING SUM(O.Qty * O.UnitPrice) > 500
ORDER BY Revenue DESC;
```

LEARN IT

SQL inside Delphi

The SQL goes into a TADOQuery as **text**. Close the query before you change the text. A query that returns records (SELECT) is run with **Open**; INSERT, UPDATE and DELETE return nothing and are run with **ExecSQL**. Values from the form are joined into the text with +: a text value needs quotes *around* it inside the SQL, and a date needs # signs. Read a field with *qryOrder['FieldName']* and move through the records with First, Eof and Next.

Delphi

```delphi
qryOrder.Close;
qryOrder.SQL.Clear;
qryOrder.SQL.Add('SELECT B.Town, SUM(O.Qty * O.UnitPrice) AS Revenue');
qryOrder.SQL.Add('FROM tblBranch B, tblOrder O');
qryOrder.SQL.Add('WHERE B.BranchID = O.BranchID');
qryOrder.SQL.Add('AND O.Product = "' + edtProduct.Text + '"');   // a text value goes in quotes
qryOrder.SQL.Add('GROUP BY B.Town');
qryOrder.Open;                                  // SELECT: Open

qryOrder.Close;
qryOrder.SQL.Text := 'UPDATE tblOrder SET UnitPrice = UnitPrice * 1.05 WHERE Product = "Bread"';
qryOrder.ExecSQL;                               // INSERT, UPDATE, DELETE: ExecSQL

qryOrder.First;                                 // reading the answer, record by record
while not qryOrder.Eof do
begin
  memOut.Lines.Add(qryOrder['Town'] + ': R' + FloatToStrF(qryOrder['Revenue'], ffFixed, 8, 2));
  qryOrder.Next;
end;
```

6.2 Run it by hand: calculated field and TOP [4]

Use tblOrder in Think First. Write the result of this query, in the order the query gives it:

```SQL
SELECT TOP 3 OrderID, Product, Qty * UnitPrice AS Total
FROM tblOrder
WHERE Qty >= 20
ORDER BY Total DESC;
```

OrderID	Product	Total

6.3 Run it by hand: a join with GROUP BY [8]

Step 1: for each branch count its orders and add up the order values (Qty × UnitPrice). Step 2: keep only revenue above 1000 and write the final order.

```SQL
SELECT B.BranchName, COUNT(O.OrderID) AS Orders, SUM(O.Qty * O.UnitPrice) AS Revenue
FROM tblBranch B, tblOrder O
WHERE B.BranchID = O.BranchID
GROUP BY B.BranchName
HAVING SUM(O.Qty * O.UnitPrice) > 1000
ORDER BY SUM(O.Qty * O.UnitPrice) DESC;
```

BranchName	Orders	Revenue (R)	Kept?	Final position
Hilltop				
Main Street				
Station Road				
Vineyard Lane				

6.4 Debug: the revenue query [5]

This query should show each **branch name** with its **revenue**, only for branches whose revenue is above R1000, highest revenue first. Find **five** errors and write the corrected query.

```sql
SELECT B.BranchName, SUM(O.Qty * O.UnitPrice) AS Revenue
FROM tblBranch B, tblOrder O
WHERE SUM(O.Qty) > 50
GROUP BY B.Town
HAVING Revenue > 1000
ORDER BY BranchName DESC;
```

6.5 Write it: two queries [8]

Write two queries. (a) Show, under the heading **AugOrders**, how many orders were placed in **August 2026**; use a date range that works for every moment of the month. (b) Show each **Town** with the **average order quantity** of its orders, rounded to 1 decimal and headed **AvgQty**, only for towns whose average is **above 25**.

6.6 Write it: change the data [5]

Write **three** separate statements: (a) add order 13 for branch 2 on 18 September 2026: 24 Scones at R12.50; (b) increase the UnitPrice of every **Bread** order from **1 September 2026** onwards by R1.50; (c) delete order 7.

6.7 Write it: show the revenue per town [8]

qryOrder is a TADOQuery. Write the code that runs a query to find the **revenue of each town** (Qty × UnitPrice summed, using the join of tblBranch and tblOrder, in alphabetical order of town), adds a line such as *Bellville: R2460.00* to **memOut** for each town, and shows the name of the town with the highest revenue in **lblTop**.

UNIT 7 Paper 2 theory: internet, networks and security

Paper 2 tests Grades 10 to 12 together. This unit revises this term's theory: search and the web, networks and internet connections, sharing and remote access, security, and how computers affect society. Every written answer here is a sentence or a short paragraph.

By the end of this unit I can: (tick each box when you are sure)

- ☐ explain semantic search, mediated search and SEO
- ☐ describe what a switch, router and wireless access point do
- ☐ compare fibre, Wi-Fi, 4G/5G and satellite connections
- ☐ explain how a VPN, SSL/TLS, a digital certificate and a firewall protect data
- ☐ discuss the effects of social networking and privacy threats in full sentences

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

7.1 The search that understands you

A learner types *why does my phone battery die overnight* and the top result is a page called *Stop apps from using power in the background*. The page never uses the words "die" or "overnight". (a) Which kind of search made this possible, keyword or semantic? Explain in a sentence. (b) The school librarian looks for articles in a database that only holds reviewed academic journals. What kind of search is that, and why can she trust the results more?

LEARN IT

Write sentences, not bullets

A marker of a 3-mark question can credit only **three** points: if you write five bullets, only the first three are read for credit, and a one-word bullet such as "encrypted" rarely proves that you understand. A paragraph is read in full, so a good point in your last sentence still earns its mark, and *because* and *so* show the reasoning the marker wants. For every mark give a **point**, say **why** it is true, and use the **scenario**. Compare, for *Explain why a school should use HTTPS on its fee page [3]*: the bullets "encrypted – padlock – safe" earn at most one mark, but "HTTPS encrypts the data between the parent's browser and the school's server, so a criminal who intercepts it sees only scrambled text. The padlock shows that the school's digital certificate was checked, so the parent can trust that the page is genuine" earns all three.

LEARN IT

Search, the web and online applications

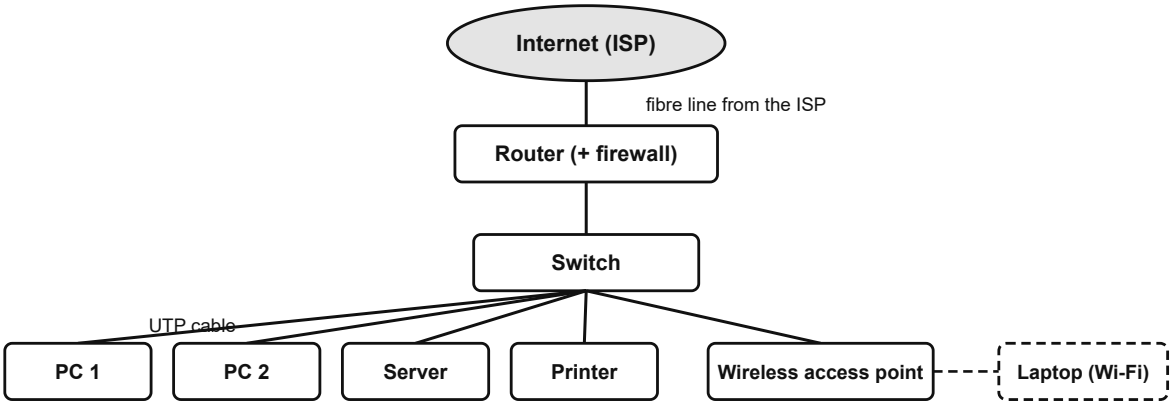
A **keyword search** matches the words typed. A **semantic search** works out the meaning and intention behind the words, using context such as earlier searches and location. A **mediated search** is done through a person or system that searches a chosen, controlled collection for you, such as Google Scholar or a library database. **SEO** (search engine optimisation) is what a website owner does to rank higher: use the words people search for, load fast, work on a phone and earn links from other sites. An **online application** runs on a server and is used through a browser, so the data lives online.

Technology	Runs where	Job
HTML	Browser	The content and structure of the page
CSS	Browser	The appearance: colours, fonts and layout
JavaScript	Browser (client-side)	Checks a form or changes the page at once, without asking the server
AJAX	Browser asks server	Fetches data in the background so only part of the page updates
PHP	Web server (server-side)	Processes forms and talks to the database
XML	Either	A text format for storing and exchanging structured data between systems
Cookie	Stored on the device	A small text file that lets a site remember you, such as a login or basket

LEARN IT

Networks, connections and sharing

A **switch** joins the devices of **one** LAN and sends data only to the device it is meant for. A **router** joins **different** networks, such as the school LAN and the Internet, and chooses the best route. A **wireless access point** (the base station for Wi-Fi) lets wireless devices join the wired network. **UTP** cable is cheap and works up to about 100 m; **fibre** carries light, is faster, goes much further and is hard to tap, but costs more.



Connection	Strength	Weakness
Fibre	Fastest, most stable, low delay	Needs a cable to the building; not everywhere
Wi-Fi	Cable-free access inside a building (the last step, not an ISP service)	Short range; shared and easier to attack
4G / 5G	Wide coverage; 5G has very low delay and handles many devices	Data is expensive and capped; signal varies
Satellite	Works almost anywhere, even without cables or towers	High delay, usually costly, weather can affect it

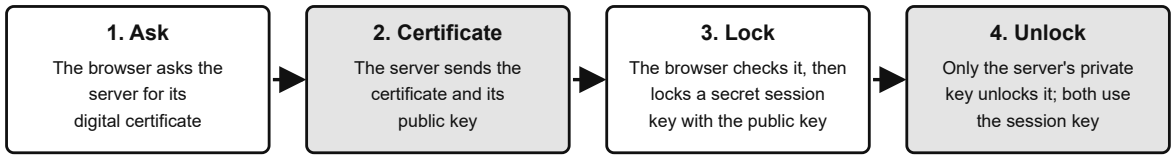
Shared folders carry **user rights** (no access, read-only, read/write, full control). Give each person the lowest right that lets them do their job, so a mistake or virus on one account cannot damage everything. **BitTorrent** is peer-to-peer sharing: a file is cut into pieces that are downloaded from many peers at once while you upload pieces to others. It spreads large legal files cheaply, but it is often used for piracy and your IP address is visible to the other peers. **Remote access** means using a computer or network from

another place: over a LAN, over the Internet, or through a **VPN**, which adds an encrypted tunnel across the public Internet so that someone at home or in a coffee shop can use the company network safely.

LEARN IT

Keeping data safe

SSL/TLS secures the connection between a browser and a web server (HTTPS and a padlock). The server has a **public key** that anyone may have and a **private key** that only it keeps; data locked with the public key can only be unlocked with the private key. A **digital certificate**, signed by a trusted certificate authority, proves that the site really belongs to the organisation. A **firewall** filters traffic into and out of a network by rules.



Grade 11 review. Symmetric encryption uses one shared key, asymmetric encryption uses a pair of keys. A **blockchain** is a chain of blocks of records, each storing a fingerprint (hash) of the block before, copied over many computers, so changing an old record breaks the chain and is noticed. **MFA** needs two or more different kinds of proof (something you know, have or are), so a stolen password alone is not enough.

LEARN IT

Computers, society and privacy

Distributed computing splits one huge problem, such as forecasting the weather for the whole country, into small pieces that many computers solve at the same time before the answers are combined: it gives the power of a supercomputer without owning one. **Social networking** helps people connect, communities organise and small businesses advertise cheaply, but it also brings cyberbullying, false information, time wasted and loss of privacy. **Cookies** record the sites you visit so that advertisers can follow you. A **GUID** is a globally unique identifier that lets a site or app recognise your device everywhere. **Anonymity** lets people speak freely but also lets them hide while they bully or commit crime. **POPIA** says that organisations may collect personal information only with a reason and consent, must keep it secure and must report breaches.

7.2 Match the term [10]

Write the term from the word bank next to each description.
Word bank: AJAX · cookie · digital certificate · distributed computing · firewall · mediated search · semantic search · switch · VPN · wireless access point

Description	Term
Works out the meaning and intention behind the words of a search	
A search through a person or system that looks only in a chosen collection, such as Google Scholar	
A small text file that a website stores on your device to remember you	
Updates part of a web page without reloading the whole page	
Sends data only to the intended device inside one LAN	
Lets wireless devices join a wired network	
An encrypted tunnel across the Internet to a private network	
Proof signed by a trusted authority that a website is genuine	
Filters network traffic according to rules	
Splits one huge job into pieces that many computers solve at the same time	

7.3 Draw the network [5]

A clinic has **8 PCs** in two rooms, a **server**, a **printer** and nurses who walk around with **tablets**. It has a fibre line from its ISP. Draw a network diagram that includes every device above plus the devices that connect them. Label every device and say where the firewall belongs.

7.4 Write: social networking and privacy [4]

Discuss one benefit and one risk of social networking, and **explain** how cookies can reduce a user's privacy.

7.5 Write: a farm without cables [4]

A game farm in the Karoo has no fibre and no cell-phone tower in range. The owner wants to run a booking website's back-end from the farm office. **Recommend** a connection and **motivate** your choice, and give one disadvantage of it. Answer in sentences.

7.6 Write: a VPN for a remote worker [4]

A bookkeeper works from home and opens files on her company's server. **Explain** how a VPN protects the data, and give one remaining risk of remote access.

7.7 Write: paying school fees online [4]

Explain how SSL/TLS, public and private keys and a digital certificate keep a parent's card number safe when paying school fees online.

7.8 Write: a shared folder and BitTorrent [4]

A school shares a *Marks* folder on its server. **Explain** why teachers should get read/write rights but learners no access, and **explain** one risk if a learner uses BitTorrent on the school network.

UNIT 8 Exam technique and your study plan

Marks are lost in the exam room for avoidable reasons: a command word is ignored, a mark allocation is not counted, or time runs out. This unit trains the habits that protect marks and helps you plan the last weeks.

By the end of this unit I can: (tick each box when you are sure)

- ☐ underline the command word and count the points a question needs
- ☐ explain why a paragraph earns more than bullet points
- ☐ plan code in comments before I write it
- ☐ work out how many minutes I can spend per mark
- ☐ write my own weekly revision timetable

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

8.1 Count the points

For each question underline the **command word**, circle the **marks**, and write how many separate points you must give and what each point should contain.

Question	Command word	Points needed	What I will write
Name TWO advantages of fibre over a 4G connection. [2]			
Explain how a firewall protects a school network. [3]			
Differentiate between a switch and a router. [4]			
Discuss the effects of social networking on privacy. [4]			

LEARN IT

Read, count, plan, answer, check

Read the whole question and underline the command word. **Count** the marks: each mark is usually one separate, correct point, so [3] means three points. **Plan** in the margin. **Answer** in sentences with *because* and *so*. **Check** that you gave as many points as there are marks. In code questions the **logic earns marks before the syntax**: write the steps as comments, then the code under each comment, and if time runs out the comments still show your thinking. In trace tables fill every row and every column: the method earns marks. Never leave a question blank.

Command word	What to do
Name / State / List	Give the answer only: a word or short phrase for each mark.
Explain	Say what and why or how: a point and its reason for each mark.
Discuss	Give both sides, benefits and drawbacks, and explain each, in a paragraph.
Differentiate	Show how two things differ, mentioning both things in every comparison.
Motivate / Justify	Choose, then give explained reasons; the word *because* must appear.
Calculate	Formula, working and answer with its unit.

8.2 Minutes per mark [6]

A paper has 150 marks and lasts 180 minutes, so each mark gets _____ minutes and a 6-mark question about _____ minutes. If you keep 15 minutes at the end to check, you have _____ minutes for the questions, which is _____ minutes per mark. The checkpoint test in this workbook has _____ marks in 150 minutes, so you have about _____ minutes for each mark.

8.3 Plan first, then code [6]

A petrol station gives a loyalty-card discount of **R0.50 per litre** when the card box **chkCard** is ticked and the customer buys **20 litres or more**. The litres are in **edtLitres** and the price per litre in **edtPrice** (both decimal numbers). Show the total in **lblTotal**, or a message if the litres are not more than 0. (a) Write your plan as comments. (b) Write the code under your comments.

8.4 My weekly revision timetable

Plan a week that you can really keep. Write the time, the topic and the task (for example *trace three recursion functions*, *one SQL past-paper question*, *write a theory answer as a paragraph*). Plan at least two Paper 2 sessions and two Paper 1 sessions, and one rest period.

Day	Time	Topic	Task I will do
Monday			
Tuesday			
Wednesday			
Thursday			
Friday			
Saturday			
Sunday			

TERM CHECKPOINT

Prelim-style checkpoint: Paper 1 practical and Paper 2 theory

Time: 150 minutes **Total:** 92 marks **Mark:** _____ / 92

No notes, no computer. Show all your working. Write Delphi code neatly and use correct indentation.

SECTION A — Recursion [10]

A1 [5]

Trace **Mix(6, 13)**. Work down to the base case, then fill in the branch and the value returned, from the bottom row up.

```Delphi
function Mix(a, b: Integer): Integer;
begin
  if b = 0 then
    Result := 0
  else if b mod 2 = 0 then
    Result := Mix(a * 2, b div 2)
  else
    Result := a + Mix(a * 2, b div 2);
end;
```

Call	a	b	Branch (odd / even / base case)	Value returned
Mix(6, 13)	6	13		
Mix(12, 6)	12	6		
Mix(24, 3)	24	3		
Mix(48, 1)	48	1		
Mix(96, 0)	96	0		

A2 [5]

Write a recursive function **SumEven(n: Integer): Integer** that returns the sum of all the even numbers from 2 up to n (n is not negative). For example SumEven(7) = 2 + 4 + 6 = 12, and SumEven(1) = 0.

SECTION B — General programming: strings, loops and decisions [9]

B1 [5]

Trace this code and write the values **after** each pass. Then write the final values of iSum and sLetters.

Delphi

```
sCode := 'RC-4172-X';
iSum := 0;
sLetters := '';
for i := 1 to Length(sCode) do
  if sCode[i] in ['0'..'9'] then
    iSum := iSum + StrToInt(sCode[i])
  else if sCode[i] <> '-' then
    sLetters := sCode[i] + sLetters;
```

i	sCode[i]	iSum	sLetters
1	R		
2	C		
3	-		
4	4		
5	1		
6	7		
7	2		
8	-		
9	X		

B2 [4]

This code should convert a mark into a symbol: 80 or more is A, 70 to 79 B, 60 to 69 C, 50 to 59 D and below 50 F. Find and correct **four** errors.

Delphi

```
iMark := edtMark.Text;
if iMark >= 80 then
  sSym := 'A';
else if iMark >= 60 then
  sSym := 'B'
else if iMark >= 50
  sSym := 'C'
else
  sSym := 'F';
lblSym.Caption := sSym;
```

SECTION C — Arrays and text files [14]

C1 [4]

arrFare holds **46, 28, 35, 19, 31**. Write the array after each pass and the number of swaps in that pass.

```Delphi
for i := 1 to 4 do
  for j := 1 to 5 - i do
    if arrFare[j] > arrFare[j + 1] then
    begin
      iTemp := arrFare[j];
      arrFare[j] := arrFare[j + 1];
      arrFare[j + 1] := iTemp;
    end;
```

Pass (i)	Array after the pass	Swaps
1		
2		
3		
4		

C2 [5]

Donations.txt has one line per donor in the form *Zinhle,250* (the amount is in rand). Write the code that reads the file into **arrDonor** and **arrAmount** (array[1..100]) and counts the donors in **iCount**. (File variable tIn; sLine and iPos are declared.)

C3 **[5]**

The arrays are now filled. Show the **total** of all donations in **lblTotal** and the **name of the biggest donor** in **lblTop**. In the same loop write every donor who gave **R500 or more** to a new file **Big.txt**, in the form *Aisha: R1200*. (File variable tOut.)

SECTION D — Object-oriented programming [9]

D1 **[4]**

A gym keeps one **TMember** object per member. Write the **type section** of the class with these private attributes: **fName** (string), **fMonths** (Integer) and **fPlan** (Char, 'B' basic or 'P' premium); and these public methods: a constructor *Create(sName: string; cPlan: Char)*; an accessor *GetName*; a mutator *AddMonths(iNum: Integer)*; *TotalPaid* (Real); and *ToString*. Add a **private auxiliary function MonthlyFee** (Real).

D2 **[3]**

Write the implementation of: the **constructor** (months start at 0); **AddMonths** (only add if iNum is more than 0); and the auxiliary **MonthlyFee** (R420 for 'P', otherwise R250).

D3 **[2]**

Write the implementation of **TotalPaid** (months × monthly fee) and **ToString**, which gives text such as *Lindiwe (P): 3 months, R1260.00*.

SECTION E — 2D arrays [8]

E1 [8]

The rainfall (mm) for 4 weeks of 7 days is stored in **arrRain[1..4, 1..7]** (row = week, column = day):

	Day 1	Day 2	Day 3	Day 4	Day 5	Day 6	Day 7
Week 1	0	3	12	0	0	8	15
Week 2	5	0	0	22	10	0	0
Week 3	0	0	7	18	25	3	0
Week 4	11	0	0	6	0	14	9

Write code to (a) total the rainfall of **week 3** in **iWeek**; (b) count in **iWet** the days with **more than 10 mm**; and (c) find the **day number** (1 to 7) with the highest rainfall **over the four weeks** and store it in **iBestDay**.

SECTION F — SQL and databases [20]

F1 [4]

A nursery keeps its plants in two tables:

tblSupplier

SupplierID	SupplierName	Town
1	Green Fingers	Paarl
2	Karoo Blooms	Worcester
3	Fynbos Direct	Stellenbosch

tblPlant

PlantID	SupplierID	PlantName	Stock	Price	DateAdded
1	1	Protea	24	45.00	2026/06/10
2	2	Aloe	60	25.00	2026/07/02
3	1	Fynbos mix	15	80.00	2026/07/15
4	3	Lavender	40	30.00	2026/07/20
5	2	Agapanthus	35	22.50	2026/08/04
6	3	Protea	12	48.00	2026/08/11
7	1	Aloe	28	27.50	2026/08/19
8	2	Lavender	17	32.00	2026/09/01

Write the result of this query:

```sql
SELECT PlantName, Stock * Price AS StockValue
FROM tblPlant
WHERE Price > 30
ORDER BY StockValue DESC;
```

PlantName	StockValue

F2 **[4]**

Write the result of this query (use both tables):

SQL

```
SELECT S.Town, COUNT(P.PlantID) AS Plants, SUM(P.Stock) AS TotalStock
FROM tblSupplier S, tblPlant P
WHERE S.SupplierID = P.SupplierID
GROUP BY S.Town
HAVING SUM(P.Stock) > 60
ORDER BY SUM(P.Stock) DESC;
```

Town	Plants	TotalStock

F3 **[4]**

Write a query that shows the **supplier name** and **plant name** of every plant added **after 1 August 2026**.

F4 **[3]**

Write **three** statements: (a) add a plant for supplier 2 named Geranium, stock 30, price R18.50; (b) increase the price of every plant from supplier 1 by 10%; (c) delete every plant with a stock below 15.

F5 **[5]**

The user sets **sedMin** to a minimum stock level. **qryPlant** is a TADOQuery. Write the code that lists the **PlantName** and **Stock** of every plant with at least that stock, in alphabetical order of name, adds a line such as *Aloe (60)* to **memOut** for each, and shows how many plants were listed in **lblCount**.

SECTION G — Theory **[22]**

Answer in full sentences or short paragraphs, never bullet points. One-word answers are only for the matching question.

G1 **[5]**

Write the term from the word bank. *Word bank: firewall · GUID · mediated search · switch · VPN*

Description	Term
A search done only in a chosen collection, such as a library's journal database	
Sends data only to the intended device inside one LAN	
An encrypted tunnel across the Internet to a company network	
Filters traffic into and out of a network according to rules	
A unique number that lets a site recognise your device everywhere	

G2 **[4]**

A rural clinic has a fibre line, a router and 8 PCs that need to share patient files. **Explain** why a **switch** is needed as well as the router, and why the clinic should give nurses read-only rights to the doctors' folder.

G3 **[4]**

Explain how SSL/TLS and a digital certificate protect a customer who pays a bill on a bank's website.

G4 **[4]**

Discuss one benefit and one risk of social networking for a learner.

G5 **[3]**

Explain how a cookie can reduce a learner's privacy and one way to limit this.

G6 **[2]**

Explain what a firewall does.

COMING UP NEXT TERM · warm-up

Your countdown to the final exam

The final exam date is _____ and there are _____ weeks left. Colour or circle each topic: **R** (red: I cannot do it yet), **A** (amber: I can do it with my notes) or **G** (green: I can do it alone and under time). Write the one thing you will do next for every red or amber topic.

Paper	Topic	Circle one	My next step
Paper 1	Recursion: trace and write	**R A G**	
Paper 1	Strings, loops and decisions	**R A G**	
Paper 1	Arrays and text files	**R A G**	
Paper 1	OOP: class and form code	**R A G**	
Paper 1	2D arrays	**R A G**	
Paper 1	SQL and database programming	**R A G**	
Paper 2	Internet, search and web technologies	**R A G**	
Paper 2	Networks, connections and remote access	**R A G**	
Paper 2	Security: SSL, keys, firewall, MFA, blockchain	**R A G**	
Paper 2	Society, social networking and privacy	**R A G**	
Paper 2	Grade 10 and 11 theory (hardware, databases, cybercrime)	**R A G**	

Now plan the last weeks. Write one Paper 1 task and one Paper 2 task for each week, and tick the past-paper question when it is done.

Week	Paper 1 task	Paper 2 task	Past paper question done
Week 4 before the exam			
Week 3			
Week 2			
Last week			

My three most common mistakes in this workbook, and how I will stop making them: